

Matr Studio — Technical Setup Documentation

Oct 7, 2026 · @Joshua

1. System architecture & topology

One Hono Worker is the only thing that touches D1 and R2; every frontend is a static build served as Workers static assets, so page views cost nothing and only API calls spend the 100,000-requests/day budget.

The design layer is the [Radian design system](#): every app and the shared `packages/ui` library are built from Radian components, which sit on Tailwind CSS and Radix primitives. Section 3 covers how Radian is wired into the monorepo.

Cloudflare now tells teams to [start new projects on Workers rather than Pages](#), so the frontends deploy with `wrangler deploy` and an `[assets]` block. Pages still works if you prefer it; only the deploy command changes.

This document assumes the domain `matr.studio` and the subdomains below. Replace them in `wrangler.toml`, `.env.production` and the CI workflow if the real domain differs.

Deployable	Path	Local port	Production host	Runtime
API	<code>packages/backend</code>	8787	<code>api.matr.studio</code>	Worker (Hono) + D1 + R2
Waitlist (phase 1)	<code>apps/waitlist</code>	3000	<code>waitlist.matr.studio</code>	Next.js static export
User app	<code>apps/user-app</code>	3001	<code>app.matr.studio</code>	Static export / SPA
Admin	<code>apps/admin</code>	3002	<code>admin.matr.studio</code>	Static export / SPA
Landing	<code>apps/landing</code>	3003	<code>matr.studio</code>	Next.js static export

```

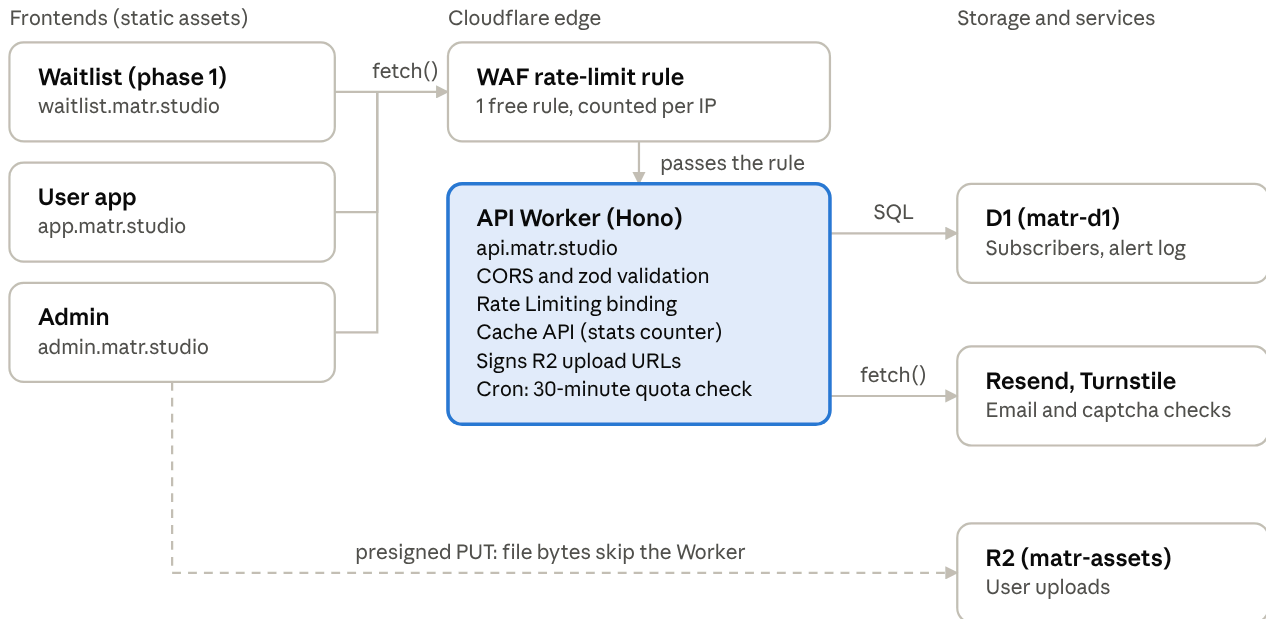
matr-studio/
├─ apps/
│  ├─ waitlist/                                # email capture + verification, phase 1
│  │  ├─ app/  components/  lib/api.ts
│  │  ├─ next.config.ts                      # output: "export"
│  │  └─ wrangler.toml                       # assets-only Worker (no `main`)
│  ├─ landing/                               # same shape as waitlist, later
│  ├─ user-app/                              # core product, later
│  └─ admin/                                # operations dashboard, later
├─ packages/
│  ├─ backend/                               # Hono Worker: the only owner of D1 and R2
│  │  └─ migrations/                         # 0001_create_waitlist_subscribers.sql,
0002_create_ops_alerts.sql
│  │  └─ src/
│  │     ├─ index.ts                         # app, CORS, scheduled() handler
│  │     ├─ env.ts                           # typed bindings
│  │     ├─ routes/                          # health.ts  waitlist.ts  uploads.ts
│  │     ├─ middleware/                      # rate-limit.ts  admin-auth.ts
│  │     └─ lib/                             # token.ts  referral.ts  cache.ts  email.ts
├─ errors.ts
│  │  └─ cron/usage-monitor.ts
│  │  └─ test/
│  │  └─ .dev.vars                           # local secrets, git-ignored
│  │  └─ wrangler.toml
│  └─ types/                                # zod schemas + API contract types
├─ api-client/                              # typed fetch wrapper used by every
frontend
│  └─ ui/                                    # Radian components (Tailwind + Radix),
cn() helper, theme tokens
├─ .github/workflows/ci-deploy.yml
├─ biome.json  tsconfig.base.json  turbo.json
├─ pnpm-workspace.yaml  package.json
└─ .gitignore

```

Three rules keep the topology simple. Only `packages/backend` declares D1 and R2 bindings. Every internal package exports TypeScript source (`"exports": { ".": "./src/index.ts" }`), so nothing needs a build step during development.

`packages/types` is the single API contract: the Worker validates with its zod schemas and the frontends import the same types.

Browsers reach D1 only through the API Worker; uploads go straight to R2



data flow · 3 frontends, 1 API Worker, D1 and R2

The frontends are static assets, so only their `fetch()` calls reach the WAF rule and then the Worker, which is the single owner of D1. The dashed path is the presigned upload that moves file bytes to R2 without touching the Worker.

2. Cloudflare free-tier resource allocation & quota strategy

The two limits that bind first are Worker requests (100,000/day) and D1 rows written (100,000/day); both reset at 00:00 UTC and, once exceeded, calls fail until the reset. Numbers below were read from Cloudflare's docs on 2026-10-07: [Workers limits](#), [Workers pricing](#), [D1 pricing](#), [D1 limits](#), [R2 pricing](#) and [WAF rate limiting](#).

Resource	Free limit	What Matr spends it on	Planning ceiling
Worker requests	100,000 / day	API calls, CORS preflights, cron runs	60,000 / day
Worker CPU time	10 ms / request (I/O wait excluded)	JSON parse, zod, HMAC	p99 under 5 ms

Resource	Free limit	What Matr spends it on	Planning ceiling
Subrequests	50 / request	Turnstile, email, webhook fetches	3 / request
Worker bundle	64 MiB uncompressed	Measured: 861 KiB (144 KiB gzip)	n/a
D1 rows read	5,000,000 / day	Stats counter, verify lookups	2,500,000 / day
D1 rows written	100,000 / day (index rows count)	Signups, verification, alert de-dupe	60,000 / day
D1 storage	5 GB account, 500 MB per database, 10 databases	One database, <code>matr-d1</code>	250 MB
D1 queries per invocation	50	1 to 2 per request	2
R2 storage	10 GB-month	User uploads (later phases)	8 GB
R2 Class A ops (writes, lists)	1,000,000 / month	Presigned PUT uploads	600,000 / month
R2 Class B ops (reads)	10,000,000 / month	Asset downloads	6,000,000 / month
R2 egress	Free	n/a	n/a
Static assets	Free and unlimited	All frontends	n/a
WAF rate-limiting rules	1 rule, IP only, 10 s window	Edge shield for <code>/api/*</code>	1 rule

The waitlist's cost per signup shows which limit is tighter. A signup writes 4 D1 rows (the subscriber row, the `email` index, the `referral_code` index and the stats counter), and the verification click writes 1 more. It also costs about 3 Worker requests: the POST, the verification GET, and an OPTIONS preflight that browsers repeat at most every 2 hours.

```

rows written per fully verified signup = 4 + 1 = 5    -> 100,000 / 5 = 20,000
signups/day
Worker requests per signup              = 1 + 1 + 1 -> 100,000 / 3 = 33,000
signups/day

```

Writes bind first, at roughly 20,000 verified signups a day. That is far above a phase-1 waitlist, but the same arithmetic is why the patterns below exist.

Cache reads at the edge

The public "people on the waitlist" number is the one endpoint anyone can hit repeatedly, so it is served from a one-row counter table and cached. A `COUNT(*)` would read every subscriber row on every miss, because D1 bills rows scanned, not rows returned.

The Workers Cache API is per data center: a response cached in one location does not exist in another, so worst-case origin reads are one per location per TTL. It also only works on custom domains, and is a silent no-op on `*.workers.dev` and in dashboard previews ([Cache API docs](#)). Test caching against `api.matr.studio`, not the `workers.dev` URL. `cache.put` accepts only GET responses and does not support `stale-while-revalidate`.

The helper in `src/lib/cache.ts` (full file in section 4) stores the JSON payload only, keyed by origin and path, and leaves CORS headers to the middleware. The stats route also sends `Cache-Control: public, max-age=60`, so browsers skip the Worker entirely for a minute.

Minimize and batch D1 writes

- Make the insert idempotent in one statement. `INSERT ... ON CONFLICT (email) DO NOTHING RETURNING id` replaces SELECT-then-INSERT: one query, no race, and a duplicate writes nothing.
- Count indexes before adding them. Every index adds a row written per insert, which is why `referral_code` and `email` are the only secondary indexes.
- Keep tokens stateless. Verification links are HMAC-signed (`src/lib/token.ts`), so there is no token table to write or expire.
- Never write on a read path. No `last_seen` updates, no per-request logging to D1; use Workers Logs for request telemetry.
- Group related statements with `db.batch()`. A batch is one round trip and one transaction, and it counts as one of the 50 queries per invocation:

```
await env.DB.batch([
  env.DB.prepare("INSERT INTO waitlist_subscribers (email, referral_code)
VALUES (?1, ?2)").bind(email, code),
  env.DB.prepare("INSERT OR IGNORE INTO ops_alerts (day, metric, level)
VALUES (?1, ?2, ?3)").bind(day, "signups", 80),
]);
```

The stats counter is a deliberate trade: one extra write per signup (a trigger in migration 0001) buys a 1-row read instead of a table scan. Drop the trigger and compute the count in a cron job if writes ever become the scarcer resource.

Upload straight to R2 with presigned URLs

The Worker signs a short-lived PUT URL and the browser sends the bytes to R2 directly, so file uploads never pass through the Worker's 128 MB memory or request budget. Presigned URLs need an [R2 API token](#) (access key ID and secret), only work on the `<account-id>.r2.cloudflarestorage.com` endpoint (custom domains are not supported), and can live from 1 second to 7 days.

`aws4fetch` (a few KB) does the signing; the full AWS SDK would blow the CPU budget at startup. Save this as `packages/backend/src/routes/uploads.ts`:

```
import { zValidator } from "@hono/zod-validator";
import { AwsClient } from "aws4fetch";
import { Hono } from "hono";
import { z } from "zod";
import type { AppEnv } from "../env";
import { apiError } from "../lib/errors";
import { requireAdmin } from "../middleware/admin-auth";

const MAX_BYTES = 5 * 1024 * 1024;
const EXT: Record<string, string> = {
  "image/png": "png",
  "image/jpeg": "jpg",
  "image/webp": "webp",
  "application/pdf": "pdf",
};

const EXPIRES_IN = 300; // seconds

const presignSchema = z.object({
  filename: z.string().min(1).max(200),
  contentType: z.enum(["image/png", "image/jpeg", "image/webp",
```

```

"application/pdf"]]),
  size: z.number().int().positive().max(MAX_BYTES),
});

export const uploads = new Hono<AppEnv>();
uploads.use("*", requireAdmin); // swap for real user auth when apps/user-app
lands

// POST /api/uploads/presign: the Worker only signs; the browser sends the
bytes straight to R2.
uploads.post("/presign", zValidator("json", presignSchema), async (c) => {
  const { contentType } = c.req.valid("json");
  const key = `uploads/${new Date().toISOString().slice(0,
10)}/${crypto.randomUUID()}.${EXT[contentType]}`;
  const headers = { "Content-Type": contentType };

  // The local R2 emulator has no S3 endpoint, so in dev the "presigned" URL
points back at this Worker.
  // (PUBLIC_API_URL, not c.req.url: wrangler dev rewrites the request host
to the production route.)
  if (c.env.ENVIRONMENT === "development") {
    return c.json({
      method: "PUT" as const,
      url: `${c.env.PUBLIC_API_URL}/api/uploads/dev/${key}`,
      key,
      headers: { ...headers, Authorization: c.req.header("Authorization") ??
"" },
      expiresIn: EXPIRES_IN,
    });
  }

  const r2 = new AwsClient({
    accessKeyId: c.env.R2_ACCESS_KEY_ID,
    secretAccessKey: c.env.R2_SECRET_ACCESS_KEY,
    service: "s3",
    region: "auto",
  });
  const target = new URL(
    `https://${c.env.R2_ACCOUNT_ID}.r2.cloudflarestorage.com/${c.env.R2_BUCKET_NAME}/${key}`,
  );
  target.searchParams.set("X-Amz-Expires", String(EXPIRES_IN));

```

```

    const signed = await r2.sign(new Request(target, { method: "PUT", headers
  })), {
    aws: { signQuery: true },
  });
  return c.json({
    method: "PUT" as const,
    url: signed.url,
    key,
    headers,
    expiresIn: EXPIRES_IN,
  });
});

// PUT /api/uploads/dev/*: development-only stand-in for the R2 S3 endpoint.
uploads.put("/dev/*", async (c) => {
  if (c.env.ENVIRONMENT !== "development") return
  c.json(apiError("not_found", "Not found."), 404);
  const key = c.req.path.replace("/api/uploads/dev/", "");
  await c.env.BUCKET.put(key, c.req.raw.body, {
    httpMetadata: { contentType: c.req.header("Content-Type") ??
  "application/octet-stream" },
  });
  return c.body(null, 200);
});

```

Two limits of this pattern are worth knowing. With `signQuery`, only the `host` header is signed (checked by running the signer), so R2 does not enforce the declared content type or size; the Worker validates both in the request, and objects over the cap should be removed by a cleanup job when uploads go live. And the browser needs bucket CORS before it can PUT. Save as `packages/backend/r2-cors.json` and apply it once:

```
{
  "rules": [
    {
      "allowed": {
        "origins": ["https://app.matr.studio", "https://admin.matr.studio",
"http://localhost:3001", "http://localhost:3002"],
        "methods": ["PUT", "GET", "HEAD"],
        "headers": ["content-type"]
      }
    }
  ]
}
```

```
pnpm --filter @matr/backend exec wrangler r2 bucket cors set matr-assets --
file r2-cors.json
```

Stay under the 10 ms CPU cap

CPU time counts only computation. Time spent waiting on `fetch()`, D1 or R2 [does not count](#), so the budget is spent on parsing, validating and crypto, not on database latency.

- Cap request bodies before parsing. The signup route uses `bodyLimit` at 4 KB; a signup is about 200 bytes.
- Use native WebCrypto for HMAC and hashing (as `token.ts` does). Do not bundle JavaScript crypto libraries, and do not hash passwords with `bcrypt` or `argon2` inside a free-plan Worker; they exceed 10 ms. When `user-app` needs sign-in, use an external identity provider or passkeys.
- Push slow work off the response path with `ctx.waitUntil()`: verification email, Turnstile follow-ups, webhooks. Waiting still happens, but it is I/O, not CPU.
- Keep dependencies lean: Hono (default router), `aws4fetch`, `zod`. The measured bundle is 861 KiB (144 KiB gzip), `zod` is 92% of it (442 of 481 KiB once minified, measured with esbuild's metafile), so `zod/mini` is the first thing to try if startup time or size ever matters.
- Measure rather than guess: watch the CPU-time quantiles in the Workers dashboard, and run `wrangler tail` while load-testing locally.

3. Local development environment & tooling

Local development needs no Cloudflare account: `wrangler dev` runs the Worker on a local runtime with emulated D1, R2 and rate-limit bindings, and state lives in `packages/backend/.wrangler/state`. Everything below was run end to end on Node 22 (typecheck, unit tests, migrations, the Worker against real HTTP calls, and the Next.js static export).

Prerequisites

Tool	Version	Why
Node.js	22 LTS or newer	Runtime for tooling and Next.js 16
pnpm	12.x, pinned by <code>packageManager</code>	Workspaces; enable with <code>corepack enable</code>
Wrangler	4.x, installed as a dev dependency	Local runtime, migrations, deploys; always run through <code>pnpm exec</code> or package scripts
Cloudflare account	Free	Only for first deploy and resource creation, not for local work

```
corepack enable
git clone <your-repo-url> matr-studio && cd matr-studio
pnpm install

# local secrets, git-ignored (template further down)
cp packages/backend/.dev.vars.example packages/backend/.dev.vars

pnpm db:migrate:local    # creates the local D1 database and applies
migrations/
pnpm dev                 # backend :8787 + every app that defines a dev script
# or just the phase-1 pair:
pnpm dev:waitlist        # backend :8787 + waitlist :3000
```

pnpm 12 refuses to run dependency install scripts until they are allowed, and Wrangler's runtime (`workerd`) and `esbuild` need theirs. That is the `allowBuilds` block in the workspace file below; without it `pnpm install` stops with `ERR_PNPM_IGNORED_BUILDS`.

Root configuration

`pnpm-workspace.yaml`:

```
packages:
  - "apps/*"
  - "packages/*"

allowBuilds:
  esbuild: true
  workerd: true
```

If pnpm rejects a package that was published within its release-age window, it prints the `minimumReleaseAgeExclude` entry to add; that setting is pnpm's supply-chain guard and is safe to leave on.

`package.json` (root):

```
{
  "name": "matr-studio",
  "private": true,
  "packageManager": "pnpm@12.9.1",
  "engines": { "node": ">=22" },
  "scripts": {
    "dev": "turbo run dev",
    "dev:waitlist": "turbo run dev --filter=@matr/waitlist --filter=@matr/backend",
    "build": "turbo run build",
    "lint": "biome check .",
    "format": "biome check --write .",
    "typecheck": "turbo run typecheck",
    "test": "turbo run test",
    "db:migrate:local": "pnpm --filter @matr/backend db:migrate:local"
  },
  "devDependencies": {
    "@biomejs/biome": "^2.5.15",
    "turbo": "^2.11.7",
    "typescript": "^7.0.2"
  }
}
```

`tsconfig.base.json`. TypeScript 7 removed `baseUrl`, so path aliases use `paths` alone:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "lib": ["ES2022"],
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noEmit": true,
    "skipLibCheck": true,
    "isolatedModules": true,
    "verbatimModuleSyntax": true,
    "resolveJsonModule": true
  }
}
```

`biome.json`. Biome handles lint and formatting in one dependency; swap in ESLint and Prettier if the team prefers them. The `tailwindDirectives` flag lets it parse Tailwind v4's `@source`:

```
{
  "$schema": "https://biomejs.dev/schemas/2.5.15/schema.json",
  "files": {
    "includes": [
      "**",
      "!**/node_modules",
      "!**/.next",
      "!**/.turbo",
      "!**/out",
      "!**/dist",
      "!**/.wrangler",
      "!**/next-env.d.ts"
    ]
  },
  "css": { "parser": { "tailwindDirectives": true } },
  "formatter": { "indentStyle": "space", "indentWidth": 2, "lineWidth": 100 },
  "linter": { "enabled": true },
  "assist": { "actions": { "source": { "organizeImports": "on" } } }
}
```

`.gitignore`:

```

node_modules
.next
out
dist
.turbo
.wrangler
.dev.vars
.env*.local
*.tsbuildinfo

```

Turborepo pipeline (`turbo.json`)

Every app and the Worker run in parallel under `turbo run dev`. Port conflicts are avoided by pinning each package's port in its own `dev` script (8787, 3000, 3001, 3002, 3003, plus the Worker inspector on 9229), not by Turborepo. `dev` tasks are `persistent` and `uncached`, and internal packages export source, so no `dev` task has to wait for a build.

```

{
  "$schema": "https://turbo.build/schema.json",
  "ui": "stream",
  "tasks": {
    "dev": {
      "cache": false,
      "persistent": true
    },
    "build": {
      "dependsOn": ["^build"],
      "env": ["NEXT_PUBLIC_*"],
      "outputs": ["out/**", ".next/**", "!next/cache/**", "dist/**"]
    },
    "typecheck": {
      "dependsOn": ["^typecheck"]
    },
    "test": {
      "dependsOn": ["^build"]
    },
    "deploy": {
      "dependsOn": ["build"],

```

```

    "cache": false
  }
}
}

```

`NEXT_PUBLIC_*` is listed under `build.env` so a change of API URL invalidates the cached static export; without it Turborepo could replay a build that still points at the old API.

Workspace packages

All internal packages are consumed as TypeScript source, so there is nothing to build before `pnpm dev`. Each package's `tsconfig.json` extends `tsconfig.base.json`; the ones that render UI add `"lib": ["ES2022", "DOM"]`.

`packages/types/package.json`:

```

{
  "name": "@matr/types",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "exports": { ".": "./src/index.ts" },
  "scripts": { "typecheck": "tsc -p tsconfig.json" },
  "dependencies": { "zod": "^4.6.5" }
}

```

`packages/api-client/package.json`:

```

{
  "name": "@matr/api-client",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "exports": { ".": "./src/index.ts" },
  "scripts": { "typecheck": "tsc -p tsconfig.json" },
  "dependencies": { "@matr/types": "workspace:*" }
}

```

`packages/backend/package.json`:

```

{
  "name": "@matr/backend",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "wrangler dev --port 8787 --inspector-port 9229 --test-scheduled",
    "build": "wrangler deploy --dry-run --outdir dist",
    "deploy": "wrangler deploy",
    "typecheck": "tsc -p tsconfig.json",
    "test": "vitest run",
    "db:migrate:local": "wrangler d1 migrations apply matr-d1 --local",
    "db:migrate:remote": "wrangler d1 migrations apply matr-d1 --remote"
  },
  "dependencies": {
    "@hono/zod-validator": "^0.9.1",
    "@matr/types": "workspace:*",
    "aws4fetch": "^1.0.20",
    "hono": "^4.13.13",
    "zod": "^4.6.5"
  },
  "devDependencies": {
    "@cloudflare/workers-types": "^5.20261007.1",
    "vitest": "^5.0.3",
    "wrangler": "^4.148.0"
  }
}

```

packages/backend/tsconfig.json:

```

{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": { "types": ["@cloudflare/workers-types"] },
  "include": ["src", "test"]
}

```

packages/ui/package.json. The dependency list is the baseline Radian components need (Radix primitives, class-variance-authority, clsx, tailwind-merge); after running the Radian CLI, reconcile it with what the CLI installed, as described in the Radian section below:

```

{
  "name": "@matr/ui",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "exports": { ".": "./src/index.ts" },
  "scripts": { "typecheck": "tsc -p tsconfig.json" },
  "dependencies": {
    "@radix-ui/react-slot": "^1.2.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "tailwind-merge": "^3.0.0"
  },
  "peerDependencies": { "react": "^19", "react-dom": "^19" },
  "devDependencies": { "@types/react": "^19.2.0", "tailwindcss": "^4.1.0" }
}

```

apps/waitlist/package.json:

```

{
  "name": "@matr/waitlist",
  "version": "0.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev --port 3000",
    "build": "next build",
    "start": "npx serve out -l 3000",
    "typecheck": "tsc -p tsconfig.json",
    "deploy": "wrangler deploy"
  },
  "dependencies": {
    "@matr/api-client": "workspace:*",
    "@matr/types": "workspace:*",
    "@matr/ui": "workspace:*",
    "next": "^16.4.0",
    "react": "^19.3.0",
    "react-dom": "^19.3.0"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4.3.3",
    "@types/node": "^26.6.4",
    "@types/react": "^19.2.0",
  }
}

```

```

    "@types/react-dom": "^19.2.0",
    "postcss": "^8.5.29",
    "tailwindcss": "^4.3.3",
    "wrangler": "^4.148.0"
  }
}

```

Worker configuration for local and production

One `wrangler.toml` serves both. Production values sit at the top level, and `.dev.vars` overrides `[vars]` during `wrangler dev` (confirmed: the local Worker reports `env: development`). That avoids duplicating every binding under an `[env.*]` block. `database_id` can stay a placeholder locally; replace it with the real ID in section 6.

`packages/backend/wrangler.toml`:

```

#:schema node_modules/wrangler/config-schema.json
name = "matr-api"
main = "src/index.ts"
compatibility_date = "2026-10-01"

workers_dev = false # production traffic must use api.matr.studio, where the
WAF rule and Cache API apply
routes = [{ pattern = "api.matr.studio", custom_domain = true }]

[vars]
ENVIRONMENT = "production"
ALLOWED_ORIGINS =
"https://waitlist.matr.studio,https://matr.studio,https://app.matr.studio,https://admin.matr.studio"
PUBLIC_WEB_URL = "https://waitlist.matr.studio"
PUBLIC_API_URL = "https://api.matr.studio"
EMAIL_FROM = "Matr Studio <hello@matr.studio>"
R2_BUCKET_NAME = "matr-assets"

[observability]
enabled = true

[[d1_databases]]
binding = "DB"
database_name = "matr-d1"
database_id = "REPLACE_WITH_ID_FROM_wrangler_d1_create"

```

```

migrations_dir = "migrations"

[[r2_buckets]]
binding = "BUCKET"
bucket_name = "matr-assets"

[[ratelimits]]
name = "SIGNUP_LIMITER"
namespace_id = "1001"
simple = { limit = 5, period = 60 }

[[ratelimits]]
name = "API_LIMITER"
namespace_id = "1002"
simple = { limit = 60, period = 60 }

[triggers]
crons = ["*/30 * * * *"]

```

Commit `packages/backend/.dev.vars.example` and copy it to the git-ignored `.dev.vars`:

```

ENVIRONMENT=development
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3001,http://localhost:
3002,http://localhost:3003
PUBLIC_WEB_URL=http://localhost:3000
PUBLIC_API_URL=http://localhost:8787
VERIFY_SECRET=dev-only-change-me
ADMIN_API_TOKEN=dev-admin-token
R2_ACCOUNT_ID=dev
R2_ACCESS_KEY_ID=dev
R2_SECRET_ACCESS_KEY=dev
CF_ACCOUNT_ID=dev
CF_API_TOKEN=dev
ALERT_WEBHOOK_URL=http://localhost:9999/hook
# Optional locally: RESEND_API_KEY= (verification email), TURNSTILE_SECRET=
(captcha)

```

Three behaviours of the local runtime caught during testing and are worth knowing:

- With a `custom_domain` route in the config, `wrangler dev` rewrites the request's host to the production hostname. Build absolute URLs from `PUBLIC_API_URL`, never from `c.req.url`.

- The local R2 emulator has no S3-compatible endpoint, so presigned URLs cannot be tested locally. The upload route returns a URL pointing back at the Worker in development (see section 2).
- The rate-limit binding works locally: the sixth signup within a minute from one IP returns 429.

To wipe local state, delete `packages/backend/.wrangler/state` and rerun `pnpm db:migrate:local`.

Design system: Radian UI

The visual layer of every Matr Studio app comes from the [Radian design system](#). Radian is an open-source React component library, set of UI blocks and Figma UI kit built on Tailwind CSS and Radix primitives, with dark mode included. It is an "open code" system: a CLI copies component source into your repository instead of installing a runtime package, so Matr owns and can edit every component.

Radian asset	Where it is used
React components (via the <code>radianui</code> CLI)	<code>packages/ui</code> , consumed by every app as <code>@matr/ui</code>
Theme tokens (brand colour, font, CSS variables)	One <code>theme.css</code> in <code>packages/ui</code> , imported by every app
Figma UI kit	Design source of truth for <code>landing</code> , <code>user-app</code> and <code>admin</code>
UI blocks (auth, web app, onboarding) at blocks.radianui.com	Starting points for <code>user-app</code> sign-in and onboarding screens

Checked with Radian CLI 0.4.0 on 2026-10-07: `init` requires a detected Next.js (or Vite) project, a Tailwind config and an `@/*` import alias, and then writes `components.json` and a `lib/utils.ts` with the `cn()` helper. Pointed at a bare library package like `packages/ui`, it stops with "could not detect a supported framework". So components are generated inside `apps/waitlist` and promoted into the shared package:

```
# 1. Give the app what the CLI checks for: Tailwind v4 and the @/* alias
pnpm --filter @matr/waitlist add -D tailwindcss @tailwindcss/postcss postcss
```

```
apps/waitlist/postcss.config.mjs:
```

```
export default { plugins: { "@tailwindcss/postcss": {} } };
```

`apps/waitlist/app/globals.css`. Tailwind v4 scans only the current project, so the shared package must be added as a source, otherwise classes used inside `@matr/ui` are not generated:

```
@import "tailwindcss";

/* Tailwind v4 only scans the current project by default. Point it at the
shared UI package. */
@source "../../packages/ui/src";
```

In `apps/waitlist/tsconfig.json` add `"paths": { "@/*": ["./*"] }` under `compilerOptions`, import `./globals.css` from `app/layout.tsx`, then generate the components:

```
cd apps/waitlist
pnpm dlx radianui@latest init --next          # prompts for brand colour and
font
pnpm dlx radianui@latest search button        # find components by name or
description
pnpm dlx radianui@latest add button input     # add what the waitlist needs
```

Promote the generated code into `packages/ui` so every app shares one copy. The `sed` step matters: generated files import `@/lib/utils`, and inside a shared package the `@/*` alias would resolve against the consuming app, not the package.

```
mkdir -p ../../packages/ui/src/components ../../packages/ui/src/lib
../../packages/ui/src/styles
git mv components/ui ../../packages/ui/src/components/ui
git mv lib/utils.ts ../../packages/ui/src/lib/utils.ts
grep -rL "@/lib/utils" ../../packages/ui/src | xargs sed -i
's#@/lib/utils#../../lib/utils#'
git rm ../../packages/ui/src/cn.ts           # replaced by lib/utils.ts
git diff package.json                       # dependencies the CLI added: move
them to packages/ui
```

Then finish the wiring by hand:

1. Replace `packages/ui/src/index.ts` with `export * from "./lib/utils";` plus one `export * from "./components/ui/<Component>";` line per generated component.

2. Move the CSS-variable and `@theme` blocks that `init` wrote into `apps/waitlist/app/globals.css` into `packages/ui/src/styles/theme.css`, and import it from each app's `globals.css` with `@import "../.../packages/ui/src/styles/theme.css";` (a relative path avoids depending on how Tailwind resolves package exports).
3. Move the dependencies the CLI added from `apps/waitlist/package.json` into `packages/ui/package.json`, then run `pnpm install`.
4. Import in any app with `import { Button } from "@matr/ui"`. Add the same `@source` line to every new app's `globals.css`.

Not verified here: the component download itself. Radian's registry (`radianui.com/r/...`) was blocked from the environment this document was prepared in, so the `init` CSS template fetch and `add` could not complete. Run the commands above once on a normal machine, review the generated files, and commit them; after that the build does not depend on the registry.

4. Backend implementation blueprint (Hono + D1 + R2)

The API is three small route modules behind one Hono app: a typed `Bindings` object, an exact-match CORS allow-list, a per-IP rate limiter, and D1 statements written to cost the fewest rows. All code in this section typechecks under strict TypeScript, the unit tests pass, and every endpoint was exercised over HTTP against `wrangler dev` (valid and duplicate signup, invalid body, rate-limit 429, CORS preflight from an allowed and a disallowed origin, verification link, upload presign and PUT).

Shared contract (`packages/types/src/index.ts`)

The zod schema is the single definition of a valid signup. The Worker parses with it; the form and `api-client` import the inferred types.

```
import { z } from "zod";

/** POST /api/waitlist request body. Shared by the Worker (validation) and
the forms (typing). */
export const waitlistSignupSchema = z.object({
  email: z.string().trim().toLowerCase().max(254).pipe(z.email()),
  referralCode: z
    .string()
    .regex(/^([0-9A-HJKMNP-TV-Z]{8})$/)
```

```

        .optional(),
    turnstileToken: z.string().max(2048).optional(),
    metadata: z
        .object({
            source: z.string().max(64).optional(),
            utmSource: z.string().max(64).optional(),
            utmCampaign: z.string().max(64).optional(),
        })
        .optional(),
    });
export type WaitlistSignupRequest = z.input<typeof waitlistSignupSchema>;

export type WaitlistSignupResponse = { ok: true };
export type WaitlistStatsResponse = { total: number };
export type HealthResponse = { ok: true; env: string; time: string };

export type PresignUploadRequest = {
    filename: string;
    contentType: string;
    size: number;
};
export type PresignUploadResponse = {
    method: "PUT";
    url: string;
    key: string;
    headers: Record<string, string>;
    expiresIn: number;
};

export type ApiErrorBody = {
    ok: false;
    error: { code: string; message: string };
};

```

Typed bindings (`packages/backend/src/env.ts`)

```

export type Bindings = {
    // Resource bindings
    DB: D1Database;
    BUCKET: R2Bucket;
    SIGNUP_LIMITER: RateLimit;
    API_LIMITER: RateLimit;
};

```

```
// Plain vars (wrangler.toml [vars], overridden locally by .dev.vars)
ENVIRONMENT: "development" | "production";
ALLOWED_ORIGINS: string; // comma-separated
PUBLIC_WEB_URL: string;
PUBLIC_API_URL: string;
EMAIL_FROM: string;
R2_BUCKET_NAME: string;

// Secrets (wrangler secret put / .dev.vars)
VERIFY_SECRET: string;
ADMIN_API_TOKEN: string;
R2_ACCOUNT_ID: string;
R2_ACCESS_KEY_ID: string;
R2_SECRET_ACCESS_KEY: string;
CF_ACCOUNT_ID: string;
CF_API_TOKEN: string; // Account Analytics: Read
ALERT_WEBHOOK_URL: string;
RESEND_API_KEY?: string;
TURNSTILE_SECRET?: string;
};

export type AppEnv = { Bindings: Bindings };
```

Entry point and CORS (`packages/backend/src/index.ts`)

CORS is an exact-match allow-list read from `ALLOWED_ORIGINS`, so adding a subdomain is a config change, not a code change. Disallowed origins get no `Access-Control-Allow-Origin` header, which makes the browser block the call. `maxAge: 7200` is the longest value Chrome honours, and it matters for quota: each preflight is a billable Worker request, so one preflight should cover two hours of calls.

```
import { Hono } from "hono";
import { cors } from "hono/cors";
import { checkUsage } from "../cron/usage-monitor";
import type { AppEnv, Bindings } from "../env";
import { apiError } from "../lib/errors";
import { health } from "../routes/health";
import { uploads } from "../routes/uploads";
import { waitlist } from "../routes/waitlist";

const app = new Hono<AppEnv>();
```

```
// CORS: exact-match allow-list from ALLOWED_ORIGINS. maxAge 7200 s is
// Chrome's ceiling; it lets one
// preflight cover two hours, because every OPTIONS request is a billable
// Worker request.
app.use("*", (c, next) => {
  const allowed = c.env.ALLOWED_ORIGINS.split(",").map((o) => o.trim());
  return cors({
    origin: (origin) => (allowed.includes(origin) ? origin : null),
    allowMethods: ["GET", "POST", "PUT", "OPTIONS"],
    allowHeaders: ["Content-Type", "Authorization"],
    maxAge: 7200,
  })(c, next);
});

app.route("/health", health);
app.route("/api/waitlist", waitlist);
app.route("/api/uploads", uploads);

app.notFound((c) => c.json(apiError("not_found", "Not found."), 404));
app.onError((err, c) => {
  console.error(err);
  return c.json(apiError("internal_error", "Something went wrong."), 500);
});

export default {
  fetch: app.fetch,
  scheduled(_controller, env, ctx) {
    ctx.waitUntil(checkUsage(env));
  },
} satisfies ExportedHandler<Bindings>;
```

If every frontend and the API sit on one zone, a Workers route such as `waitlist.matr.studio/api/*` pointing at this Worker makes calls same-origin and removes CORS and preflights entirely. The cost is one route per app and tighter coupling, so this document keeps the separate `api.matr.studio` host.

D1 schema migrations

Migrations are plain SQL files in `packages/backend/migrations/`, applied in order by Wrangler and tracked in a `d1_migrations` table. Create the next one with `pnpm --filter @matr/backend exec wrangler d1 migrations create matr-d1 <name>`.

migrations/0001_create_waitlist_subscribers.sql:

```

-- Waitlist subscribers.
-- Write cost per signup: 1 table row + 1 index row (email) + 1 index row
(referral_code) = 3 rows written.
CREATE TABLE waitlist_subscribers (
  id          INTEGER PRIMARY KEY,          -- rowid alias: no extra
index
  email       TEXT    NOT NULL,
  status      TEXT    NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'verified', 'unsubscribed',
'bounce'd')),
  referral_code TEXT    NOT NULL,
  referred_by  TEXT,                        -- referral_code of the
referrer, unchecked on write
  metadata     TEXT    NOT NULL DEFAULT '{}' CHECK (json_valid(metadata)),
  created_at   INTEGER NOT NULL DEFAULT (unixepoch()),
  updated_at   INTEGER NOT NULL DEFAULT (unixepoch())
);

CREATE UNIQUE INDEX idx_waitlist_email ON waitlist_subscribers (email);
CREATE UNIQUE INDEX idx_waitlist_referral_code ON waitlist_subscribers
(referral_code);

-- One-row counter so the public "N people joined" number costs 1 row read
instead of a table scan.
-- Costs 1 extra row written per signup (4 total).
CREATE TABLE waitlist_stats (
  id    INTEGER PRIMARY KEY CHECK (id = 1),
  total INTEGER NOT NULL DEFAULT 0
);
INSERT INTO waitlist_stats (id, total) VALUES (1, 0);

CREATE TRIGGER trg_waitlist_count
AFTER INSERT ON waitlist_subscribers
BEGIN
  UPDATE waitlist_stats SET total = total + 1 WHERE id = 1;
END;

```

migrations/0002_create_ops_alerts.sql:

```
-- De-duplicates quota alerts: one row per (UTC day, metric, threshold).
CREATE TABLE ops_alerts (
  day      TEXT      NOT NULL,    -- YYYY-MM-DD (UTC)
  metric   TEXT      NOT NULL,    -- requests | rowsRead | rowsWritten
  level    INTEGER NOT NULL,      -- 80 | 95
  sent_at  INTEGER NOT NULL DEFAULT (unixepoch()),
  PRIMARY KEY (day, metric, level)
) WITHOUT ROWID;
```

The schema columns match the brief (id, email, status, metadata, created_at) plus referral_code, referred_by and updated_at for the referral and verification flow. status is constrained to four values, metadata must be valid JSON, and email is stored lowercased because the zod schema normalises it before the insert. Both migrations, including the trigger, apply cleanly to the local D1 emulator.

Endpoints

Endpoint	Purpose	Protection	D1 cost
GET /health	Liveness; touches no storage	None	0
GET /health/db	Manual dependency check	None	1 query
POST /api/waitlist	Join the waitlist	5 per minute per IP, 4 KB body cap, optional Turnstile	1 query, 4 rows written
GET /api/waitlist/verify	Confirm an email from the link	60 per minute per IP	1 row written
GET /api/waitlist/stats	Public "N joined" counter	60 per minute per IP, 60 s edge and browser cache	At most 1 row read
POST /api/uploads/presign	Sign an R2 upload (section 2)	Bearer token	0

packages/backend/src/routes/waitlist.ts. The signup answers 202 { ok: true } for new and existing emails alike, so the endpoint cannot be used to find out who is on the list. The verification email is sent from waitUntil, so the visitor never waits for the email

provider.

```
import { zValidator } from "@hono/zod-validator";
import { waitlistSignupSchema } from "@matr/types";
import { Hono } from "hono";
import { bodyLimit } from "hono/body-limit";
import type { AppEnv } from "../env";
import { cachedJson } from "../lib/cache";
import { sendVerificationEmail } from "../lib/email";
import { apiError } from "../lib/errors";
import { generateReferralCode } from "../lib/referral";
import { verifyVerifyToken } from "../lib/token";
import { rateLimit } from "../middleware/rate-limit";

export const waitlist = new Hono<AppEnv>();

async function turnstileOk(secret: string, token: string | undefined, ip:
string | undefined) {
  if (!token) return false;
  const res = await
fetch("https://challenges.cloudflare.com/turnstile/v0/siteverify", {
  method: "POST",
  body: new URLSearchParams({ secret, response: token, ...(ip ? { remoteip:
ip } : {}) }),
});
  return ((await res.json()) as { success?: boolean }).success === true;
}

// POST /api/waitlist
waitlist.post(
  "/",
  rateLimit("SIGNUP_LIMITER"),
  bodyLimit({
    maxSize: 4 * 1024, // a signup is ~200 bytes; refuse anything bigger
    before parsing
    onError: (c) => c.json(apiError("payload_too_large", "Request body too
large."), 413),
  }),
  zValidator("json", waitlistSignupSchema, (result, c) => {
    if (!result.success) {
      return c.json(apiError("invalid_request", "Check the email address and
try again."), 400);
    }
  })
);
```

```

    }),
    async (c) => {
      const body = c.req.valid("json");

      if (c.env.TURNSTILE_SECRET) {
        const ok = await turnstileOk(
          c.env.TURNSTILE_SECRET,
          body.turnstileToken,
          c.req.header("cf-connecting-ip"),
        );
        if (!ok) return c.json(apiError("captcha_failed", "Verification failed. Please retry."), 400);
      }

      // One statement, one write path: INSERT ... ON CONFLICT DO NOTHING + RETURNING.
      // No SELECT-then-INSERT race and no extra read. A duplicate returns no row.
      const inserted = await c.env.DB.prepare(
        `INSERT INTO waitlist_subscribers (email, referral_code, referred_by, metadata)
        VALUES (?1, ?2, ?3, ?4)
        ON CONFLICT (email) DO NOTHING
        RETURNING id`,
      )
      .bind(
        body.email,
        generateReferralCode(),
        body.referralCode ?? null,
        JSON.stringify(body.metadata ?? {}),
      )
      .first<{ id: number }>();

      if (inserted) {
        c.executionCtx.waitUntil(sendVerificationEmail(c.env, body.email));
      }

      // Same response for new and existing emails: the endpoint cannot be used to enumerate subscribers.
      return c.json({ ok: true as const }, 202);
    },
  );

```

```
// GET /api/waitlist/verify?token=...
waitlist.get("/verify", rateLimit("API_LIMITER"), async (c) => {
  const email = await verifyVerifyToken(c.env.VERIFY_SECRET,
c.req.query("token") ?? "");
  if (!email) return c.redirect(`${c.env.PUBLIC_WEB_URL}/verify-failed`,
302);

  await c.env.DB.prepare(
    `UPDATE waitlist_subscribers
      SET status = 'verified', updated_at = unixepoch()
      WHERE email = ?1 AND status = 'pending'`,
  )
    .bind(email)
    .run();

  return c.redirect(`${c.env.PUBLIC_WEB_URL}/verified`, 302);
});

// GET /api/waitlist/stats: public counter, 1 row read on a miss, 0 on a hit.
waitlist.get("/stats", rateLimit("API_LIMITER"), async (c) => {
  const data = await cachedJson(c, 60, async () => {
    const row = await c.env.DB.prepare("SELECT total FROM waitlist_stats
WHERE id = 1").first<{
      total: number;
    }>();
    return { total: row?.total ?? 0 };
  });
  c.header("Cache-Control", "public, max-age=60"); // let browsers cache it
  too
  return c.json(data);
});
```

`packages/backend/src/routes/health.ts`. Point uptime monitors at `/health`: an external monitor polling every minute is 1,440 requests a day, about 1.4% of the Worker quota, and it should not also spend D1 reads.

```
import { Hono } from "hono";
import type { AppEnv } from "../env";

export const health = new Hono<AppEnv>();

// Does not touch D1 or R2, so an uptime monitor polling it burns zero
// database quota.
health.get("/", (c) =>
  c.json({ ok: true as const, env: c.env.ENVIRONMENT, time: new
    Date().toISOString() }),
);

// Deliberate dependency check. Point monitors at "/health", and hit this one
// by hand.
health.get("/db", async (c) => {
  await c.env.DB.prepare("SELECT 1").first();
  return c.json({ ok: true as const });
});
```

`packages/backend/src/middleware/rate-limit.ts`. The key combines path and client IP, so a burst on one endpoint does not lock a visitor out of the others:

```
import type { MiddlewareHandler } from "hono";
import type { AppEnv } from "../env";
import { apiError } from "../lib/errors";

/** Per-IP limit using the Workers Rate Limiting binding (configured in
  wrangler.toml). */
export const rateLimit =
  (binding: "SIGNUP_LIMITER" | "API_LIMITER"): MiddlewareHandler<AppEnv> =>
    async (c, next) => {
      const ip = c.req.header("cf-connecting-ip") ?? "unknown";
      const { success } = await c.env[binding].limit({ key:
        `${c.req.path}:${ip}` });
      if (!success) {
        return c.json(apiError("rate_limited", "Too many requests. Try again
          shortly."), 429, {
          "Retry-After": "60",
        });
      }
      await next();
    };
};
```

Supporting modules

`src/lib/errors.ts`, `src/lib/referral.ts` and `src/lib/token.ts`. Tokens are HMAC-SHA-256 signed with WebCrypto, which runs natively and costs microseconds of CPU; `crypto.subtle.verify` compares signatures in constant time.

```
import type { ApiErrorBody } from "@matr/types";

export const apiError = (code: string, message: string): ApiErrorBody => ({
  ok: false,
  error: { code, message },
});
```

```
// Crockford base32 (no I, L, O, U): 8 chars = 40 bits, collisions are caught
// by the UNIQUE index.
const ALPHABET = "0123456789ABCDEFGHIJKLMNPQRSTUVWXYZ";

export function generateReferralCode(): string {
  const bytes = crypto.getRandomValues(new Uint8Array(8));
  return Array.from(bytes, (b) => ALPHABET[b % 32]).join("");
}
```

```
// Stateless, HMAC-signed verification tokens: no DB row to write or read
// until the user clicks the link.
const enc = new TextEncoder();
const dec = new TextDecoder();

const b64u = {
  encode(bytes: Uint8Array): string {
    let s = "";
    for (const b of bytes) s += String.fromCharCode(b);
    return btoa(s).replaceAll("+", "-").replaceAll("/", "_").replaceAll("=",
    "");
  },
  decode(str: string): Uint8Array {
    const pad = "=".repeat((4 - (str.length % 4)) % 4);
    const bin = atob(str.replaceAll("-", "+").replaceAll("_", "/") + pad);
    return Uint8Array.from(bin, (c) => c.charCodeAt(0));
  },
};

const importKey = (secret: string, usage: "sign" | "verify") =>
```

```

    crypto.subtle.importKey("raw", enc.encode(secret), { name: "HMAC", hash:
"SHA-256" }, false, [
      usage,
    ]);

export async function signVerifyToken(
  secret: string,
  email: string,
  ttlSeconds = 7 * 24 * 3600,
): Promise<string> {
  const exp = Math.floor(Date.now() / 1000) + ttlSeconds;
  const payload = `${b64u.encode(enc.encode(email))}.${exp}`;
  const sig = await crypto.subtle.sign(
    "HMAC",
    await importKey(secret, "sign"),
    enc.encode(payload),
  );
  return `${payload}.${b64u.encode(new Uint8Array(sig))}`;
}

/** Returns the email if the token is authentic and unexpired, otherwise
null. */
export async function verifyVerifyToken(secret: string, token: string):
Promise<string | null> {
  const [emailPart, expPart, sigPart] = token.split(".");
  if (!emailPart || !expPart || !sigPart) return null;
  try {
    const ok = await crypto.subtle.verify(
      "HMAC",
      await importKey(secret, "verify"),
      b64u.decode(sigPart),
      enc.encode(`${emailPart}.${expPart}`),
    );
    if (!ok || Number(expPart) < Math.floor(Date.now() / 1000)) return null;
    return dec.decode(b64u.decode(emailPart));
  } catch {
    return null;
  }
}

```

`src/lib/email.ts` sends the confirmation through [Resend](#) from `waitUntil`; it is skipped when `RESEND_API_KEY` is unset, so local signups work without an email provider. Swap in any provider with an HTTP API:

```
import type { Bindings } from "../env";
import { signVerifyToken } from "./token";

/** Fire-and-forget from ctx.waitUntil(): the network wait does not count
toward the 10 ms CPU budget. */
export async function sendVerificationEmail(env: Bindings, email: string):
Promise<void> {
  if (!env.RESEND_API_KEY) return; // not configured (e.g. local development)
  const token = await signVerifyToken(env.VERIFY_SECRET, email);
  const link = `${env.PUBLIC_API_URL}/api/waitlist/verify?
token=${encodeURIComponent(token)}`;

  const res = await fetch("https://api.resend.com/emails", {
    method: "POST",
    headers: {
      Authorization: `Bearer ${env.RESEND_API_KEY}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      from: env.EMAIL_FROM,
      to: email,
      subject: "Confirm your spot on the Matr Studio waitlist",
      text: `Confirm your email to hold your place: ${link}`,
    }),
  });
  if (!res.ok) console.error("verification email failed", res.status);
}
```

`src/lib/cache.ts` is the edge-cache helper described in section 2:

```
import type { Context } from "hono";
import type { AppEnv } from "../env";

/**
 * Edge-cache a JSON payload with the Workers Cache API.
 * - Only active on custom domains (a no-op on *.workers.dev).
 * - The cache is per data center, so worst case = (ttl-spaced) one origin
read per colo.
 * - Stores the payload only, never CORS headers; those are added per request
by the cors middleware.
 */
export async function cachedJson<T>(<T>)
```

```

    c: Context<AppEnv>,
    ttlSeconds: number,
    load: () => Promise<T>,
  ): Promise<T> {
    const url = new URL(c.req.url);
    const cacheKey = new Request(`${url.origin}${url.pathname}`); // drop query
string and headers from the key
    const cache = caches.default;

    const hit = await cache.match(cacheKey);
    if (hit) return hit.json<T>();

    const data = await load();
    c.executionCtx.waitUntil(
      cache.put(
        cacheKey,
        new Response(JSON.stringify(data), {
          headers: {
            "Content-Type": "application/json",
            "Cache-Control": `public, max-age=${ttlSeconds}`,
          },
        }),
      ),
    );
    return data;
  }

```

`src/middleware/admin-auth.ts` guards the upload route until `user-app` has real authentication. It compares the bearer token in constant time and is meant to be replaced, not extended:

```
import type { MiddlewareHandler } from "hono";
import type { AppEnv } from "../env";
import { apiError } from "../lib/errors";

/** Placeholder until user-app auth exists: a shared bearer token held in a
Worker secret. */
export const requireAdmin: MiddlewareHandler<AppEnv> = async (c, next) => {
  const presented = new TextEncoder().encode(
    (c.req.header("Authorization") ?? "").replace(/^Bearer /, ""),
  );
  const expected = new TextEncoder().encode(c.env.ADMIN_API_TOKEN);
  const ok =
    presented.byteLength === expected.byteLength &&
    crypto.subtle.timingSafeEqual(presented, expected);
  if (!ok) return c.json(apiError("unauthorized", "Missing or invalid
credentials."), 401);
  await next();
};
```

Try it locally

With `pnpm dev:waitlist` running and migrations applied:

```
curl -i localhost:8787/health
# 200 {"ok":true,"env":"development","time":"..."}

curl -i -X POST localhost:8787/api/waitlist \
  -H 'Origin: http://localhost:3000' -H 'Content-Type: application/json' \
  -d '{"email":"Test@example.com","metadata":{"source":"curl"}}'
# 202 {"ok":true} with Access-Control-Allow-Origin: http://localhost:3000

curl -s -X POST localhost:8787/api/waitlist -H 'Content-Type:
application/json' -d '{"email":"nope"}'
# 400 {"ok":false,"error":{"code":"invalid_request",...}}

curl -s localhost:8787/api/waitlist/stats
# {"total":1} (a duplicate signup does not increase it)

pnpm --filter @matr/backend exec wrangler d1 execute matr-d1 --local \
  --command "SELECT id, email, status, referral_code FROM
waitlist_subscribers"
```

The email is stored as `test@example.com`, lowercased by the schema. Sending more than five signups from one IP inside a minute returns `429`, and a preflight from an origin outside `ALLOWED_ORIGINS` comes back without an `Access-Control-Allow-Origin` header.

The unit tests (`pnpm test`) cover token signing, tampering and expiry, and the signup schema. Save as `packages/backend/test/token.test.ts`:

```
import { waitlistSignupSchema } from "@matr/types";
import { describe, expect, it } from "vitest";
import { signVerifyToken, verifyVerifyToken } from "../src/lib/token";

describe("verification tokens", () => {
  it("round-trips an email", async () => {
    const token = await signVerifyToken("secret", "a@example.com");
    expect(await verifyVerifyToken("secret", token)).toBe("a@example.com");
  });

  it("rejects a wrong secret, a tampered payload and an expired token", async
  () => {
    const token = await signVerifyToken("secret", "a@example.com");
    expect(await verifyVerifyToken("other", token)).toBeNull();
    expect(await verifyVerifyToken("secret", `x${token}`)).toBeNull();
    const expired = await signVerifyToken("secret", "a@example.com", -10);
    expect(await verifyVerifyToken("secret", expired)).toBeNull();
  });
});

describe("waitlistSignupSchema", () => {
  it("lowercases and trims the email", () => {
    const parsed = waitlistSignupSchema.parse({ email: " Ada@Example.COM "
  });
    expect(parsed.email).toBe("ada@example.com");
  });

  it("rejects malformed emails and referral codes", () => {
    expect(waitlistSignupSchema.safeParse({ email: "nope"
  }).success).toBe(false);
    expect(waitlistSignupSchema.safeParse({ email: "a@b.co", referralCode:
  "lower" }).success).toBe(
```

```

        false,
    );
  });
});

```

The usage monitor imported by `index.ts` (`src/cron/usage-monitor.ts`) is in section 7.

5. Frontend integration pattern

Every frontend talks to the API through one typed client, `@matr/api-client`, and the only thing that changes between local and production is a single build-time variable, `NEXT_PUBLIC_API_URL`.

A dedicated package is used instead of putting the client in `packages/ui` because the client has no React or CSS dependency: the admin app, a future mobile shell or a Node script can import it unchanged. It depends only on `@matr/types`, so a contract change in the Worker is a compile error in every caller.

The shared client (`packages/api-client/src/index.ts`)

`ApiClientError` carries the HTTP status and the API's error `code`, so callers branch on `rate_limited` or `invalid_request` rather than parsing messages. The optional `headers` callback is how the admin app will attach a bearer token.

```

import type {
  ApiErrorBody,
  HealthResponse,
  PresignUploadRequest,
  PresignUploadResponse,
  WaitlistSignupRequest,
  WaitlistSignupResponse,
  WaitlistStatsResponse,
} from "@matr/types";

export class ApiClientError extends Error {
  constructor(
    readonly status: number,
    readonly code: string,
    message: string,
  ) {
    super(message);
    this.name = "ApiClientError";
  }
}

```

```

    }
  }

export type ApiClientOptions = {
  /** e.g. "http://localhost:8787" in development, "https://api.matr.studio"
  in production. */
  baseUrl: string;
  /** Override for tests or server-side use. */
  fetch?: typeof fetch;
  /** Extra headers, e.g. a bearer token for the admin app. */
  headers?: () => Record<string, string>;
};

export function createApiClient(options: ApiClientOptions) {
  const baseUrl = options.baseUrl.replace(/\/+$/, "");
  const doFetch = options.fetch ?? fetch;

  async function request<T>(path: string, init: RequestInit = {}): Promise<T>
  {
    const res = await doFetch(`${baseUrl}${path}`, {
      ...init,
      headers: {
        Accept: "application/json",
        ...(init.body ? { "Content-Type": "application/json" } : {}),
        ...options.headers?(),
        ...init.headers,
      },
    });

    if (!res.ok) {
      const body = (await res.json()).catch(() => null) as ApiErrorBody |
null;
      throw new ApiClientError(
        res.status,
        body?.error.code ?? "http_error",
        body?.error.message ?? `Request failed with ${res.status}`,
      );
    }
    return (await res.json()) as T;
  }

  return {
    health: () => request<HealthResponse>("/health"),
  }
}

```

```

waitlist: {
  join: (body: WaitlistSignupRequest) =>
    request<WaitlistSignupResponse>("/api/waitlist", {
      method: "POST",
      body: JSON.stringify(body),
    }),
  stats: () => request<WaitlistStatsResponse>("/api/waitlist/stats"),
},
uploads: {
  presign: (body: PresignUploadRequest) =>
    request<PresignUploadResponse>("/api/uploads/presign", {
      method: "POST",
      body: JSON.stringify(body),
    }),
},
};
}

export type ApiClient = ReturnType<typeof createApiClient>;

```

Wiring the waitlist app

The waitlist is a Next.js static export. Pages are plain files served by Workers static assets, and only the browser's `fetch` to the API counts against the Worker quota.

`apps/waitlist/next.config.ts`:

```

import type { NextConfig } from "next";

const config: NextConfig = {
  // Static export: pages are served as Workers static assets, which are free
  // and unlimited,
  // so page views never consume the 100k/day Worker request budget.
  output: "export",
  transpilePackages: ["@matr/ui", "@matr/types", "@matr/api-client"],
  images: { unoptimized: true },
};

export default config;

```

Next.js reads `.env.development` for `next dev` and `.env.production` for `next build` (the build log lists `.env.production` as the loaded environment file), so the two files below are the whole local-versus-production switch:

```
# apps/waitlist/.env.development
NEXT_PUBLIC_API_URL=http://localhost:8787

# apps/waitlist/.env.production
NEXT_PUBLIC_API_URL=https://api.matr.studio
```

`apps/waitlist/lib/api.ts` builds the one client instance. The value is inlined into the JavaScript at build time, which is why `turbo.json` lists `NEXT_PUBLIC_*` as a build input:

```
import { createApiClient } from "@matr/api-client";

// Inlined at build time by Next.js. Dev: http://localhost:8787, prod:
https://api.matr.studio
export const api = createApiClient({
  baseUrl: process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8787",
});
```

`apps/waitlist/components/waitlist-form.tsx`:

```
"use client";

import { ApiClientError } from "@matr/api-client";
import { type FormEvent, useState } from "react";
import { api } from "../lib/api";

type State = "idle" | "submitting" | "done" | "error";

export function WaitlistForm({ referralCode }: { referralCode?: string }) {
  const [state, setState] = useState<State>("idle");
  const [message, setMessage] = useState("");

  async function onSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    const email = String(new FormData(event.currentTarget).get("email")) ??
    "");
    setState("submitting");
    try {
      await api.waitlist.join({ email, referralCode, metadata: { source:
```

```

"waitlist-site" } });
  setState("done");
} catch (err) {
  setState("error");
  setMessage(
    err instanceof ApiClientError && err.code === "rate_limited"
      ? "Too many attempts. Please wait a minute."
      : "Could not join the waitlist. Check your email and try again.",
  );
}
}

if (state === "done") return <p>Check your inbox to confirm your spot.</p>;

return (
  <form onSubmit={onSubmit}>
    <input
      name="email"
      type="email"
      required
      placeholder="you@company.com"
      autoComplete="email"
    />
    <button type="submit" disabled={state === "submitting"}>
      {state === "submitting" ? "Joining..." : "Join the waitlist"}
    </button>
    {state === "error" && <p role="alert">{message}</p>}
  </form>
);
}

```

`apps/waitlist/app/page.tsx` and `app/layout.tsx` are the minimal shell. Once the Radian components exist in `@matr/ui` (section 3), swap the `<input>` and `<button>` for Radian's `Input` and `Button`; the logic does not change.

```
// app/page.tsx
import { WaitlistForm } from "../components/waitlist-form";

export default function Page() {
  return (
    <main>
      <h1>Matr Studio</h1>
      <WaitlistForm />
    </main>
  );
}
```

```
// app/layout.tsx
import type { ReactNode } from "react";
import "../globals.css";

export const metadata = { title: "Matr Studio: join the waitlist" };

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

The verification redirects land on two static pages, `app/verified/page.tsx` and `app/verify-failed/page.tsx`; each is a few lines of JSX with a heading. To support referral links such as `?ref=ABCD2345`, read the code from `window.location.search` inside a `useEffect` and pass it as `referralCode`. Avoid `useSearchParams` here: in a static export it forces a Suspense boundary around the page.

Local versus production

	Local	Production
Page host	<code>next dev</code> on <code>localhost:3000</code>	Static assets at <code>waitlist.matr.studio</code>
API base URL	<code>http://localhost:8787</code> (<code>.env.development</code>)	<code>https://api.matr.studio</code> (<code>.env.production</code>)

	Local	Production
Allowed CORS origins	<code>ALLOWED_ORIGINS</code> in <code>.dev.vars</code>	<code>[vars]</code> in <code>wrangler.toml</code>
D1 and R2	Emulated in <code>.wrangler/state</code>	Cloudflare D1 and R2
Verification email	Skipped without <code>RESEND_API_KEY</code>	Sent through Resend
Edge cache	Not exercised	Active on <code>api.matr.studio</code> only

The browser sends a preflight on the first cross-origin POST because the body is JSON. The API answers it with `Access-Control-Max-Age: 7200`, so each visitor pays one extra request per two hours, which is already counted in the per-signup budget in section 2.

The same client serves the other apps. `user-app` and `admin` create their own instance with a `headers` callback that returns the session token, add their origin to `ALLOWED_ORIGINS`, and import the same types.

6. Deployment pipeline & secrets management

Deployment is a one-time resource setup done by hand with Wrangler, followed by a GitHub Actions workflow that applies migrations, deploys the Worker, then deploys each frontend on every push to `main`.

Three account prerequisites decide whether the steps below work. The domain (assumed `matr.studio`) must be an active zone on your Cloudflare account, because custom domains, the Cache API and WAF rules all need one. R2 needs an [R2 subscription activated through the dashboard checkout](#), which asks for a payment method even though the monthly allowance is free; R2 has no hard cap, so the quota alerts in section 7 are your only brake. And D1 and Workers need nothing beyond a free account.

One-time resource setup

Run these from the repo root. Wrangler is a dev dependency, so `pnpm --filter @matr/backend exec wrangler` uses the pinned version.

1. Authenticate for manual commands (CI uses an API token instead):

```
pnpm --filter @matr/backend exec wrangler login
```

2. Create the D1 database. The output includes a `database_id`; paste it over the placeholder in `packages/backend/wrangler.toml`:

```
pnpm --filter @matr/backend exec wrangler d1 create matr-d1
```

3. Apply the migrations to the remote database:

```
pnpm --filter @matr/backend db:migrate:remote
```

4. Create the R2 bucket and its CORS policy (the policy file is in section 2):

```
pnpm --filter @matr/backend exec wrangler r2 bucket create matr-assets
pnpm --filter @matr/backend exec wrangler r2 bucket cors set matr-assets -
-file r2-cors.json
```

5. Create an R2 API token for presigned URLs: dashboard, R2, Manage API tokens, permission Object Read & Write, scoped to the `matr-assets` bucket. Keep the access key ID and secret for step 6, and note your account ID.
6. Set the secrets (table below). Each command prompts for the value, so nothing lands in shell history:

```
cd packages/backend
pnpm exec wrangler secret put VERIFY_SECRET          # openssl rand -base64
32
pnpm exec wrangler secret put ADMIN_API_TOKEN        # openssl rand -base64
32
pnpm exec wrangler secret put R2_ACCOUNT_ID
pnpm exec wrangler secret put R2_ACCESS_KEY_ID
pnpm exec wrangler secret put R2_SECRET_ACCESS_KEY
pnpm exec wrangler secret put CF_ACCOUNT_ID
pnpm exec wrangler secret put CF_API_TOKEN           # Account Analytics:
Read only
pnpm exec wrangler secret put ALERT_WEBHOOK_URL      # Slack or Discord
incoming webhook
pnpm exec wrangler secret put RESEND_API_KEY         # optional until email
is live
pnpm exec wrangler secret put TURNSTILE_SECRET       # optional until the
captcha is added
```

To load many at once, put them in a git-ignored JSON file and run `pnpm exec wrangler secret bulk .secrets.production.json` (up to 100 per command; add that filename to `.gitignore`).

7. Deploy the API, then the waitlist. The first deploy attaches `api.matr.studio` and registers the cron trigger:

```
pnpm --filter @matr/backend deploy
pnpm turbo run build --filter=@matr/waitlist
pnpm --filter @matr/waitlist deploy
```

8. Smoke-test: `curl -i https://api.matr.studio/health` should return `200`, and a signup from `https://waitlist.matr.studio` should appear in `wrangler d1 execute matr-d1 --remote --command "SELECT COUNT(*) FROM waitlist_subscribers"`.

Variables versus secrets

Name	Kind	Where it lives	Notes
<code>ENVIRONMENT</code> , <code>ALLOWED_ORIGINS</code> , <code>PUBLIC_WEB_URL</code> , <code>PUBLIC_API_URL</code> , <code>EMAIL_FROM</code> , <code>R2_BUCKET_NAME</code>	Variable	<code>[vars]</code> in <code>wrangler.toml</code>	Not sensitive; visible in the repo and reviewed in PRs
<code>VERIFY_SECRET</code>	Secret	<code>wrangler secret put</code>	Signs verification links; rotating it invalidates links already sent
<code>ADMIN_API_TOKEN</code>	Secret	<code>wrangler secret put</code>	Interim bearer token for upload signing
<code>R2_ACCESS_KEY_ID</code> , <code>R2_SECRET_ACCESS_KEY</code>	Secret	<code>wrangler secret put</code>	Bucket-scoped R2 token
<code>R2_ACCOUNT_ID</code> , <code>CF_ACCOUNT_ID</code>	Secret	<code>wrangler secret put</code>	Identifiers, not credentials; kept out of <code>wrangler.toml</code> so the repo has no account-specific values
<code>CF_API_TOKEN</code>	Secret	<code>wrangler secret put</code>	Read-only analytics token used by the usage monitor

Name	Kind	Where it lives	Notes
<code>ALERT_WEBHOOK_URL</code> , <code>RESEND_API_KEY</code> , <code>TURNSTILE_SECRET</code>	Secret	<code>wrangler secret</code> <code>put</code>	Third-party credentials
<code>CLOUDFLARE_API_TOKEN</code> , <code>CLOUDFLARE_ACCOUNT_ID</code>	CI secret	GitHub environment <code>production</code>	Deploy credentials; never read by the Worker

Secrets are encrypted at rest, never appear in `wrangler.toml` , and are not readable back through Wrangler or the dashboard after they are set. Locally they come from the git-ignored `.dev.vars` . Use three separate Cloudflare tokens with the narrowest scope each: one for CI deploys, one read-only analytics token for `CF_API_TOKEN` , and one R2 object-read/write token.

Production should not be reachable on the `workers.dev` hostname. Requests there bypass the zone's WAF rule and the Cache API (section 2), so set `workers_dev = false` in `packages/backend/wrangler.toml` once the custom domain works.

GitHub Actions workflow

Save as `.github/workflows/ci-deploy.yml` . Pull requests run lint, typecheck, tests and build; only pushes to `main` deploy. Migrations run before the Worker so the old code keeps working until the new code is live, which means each migration must be backward compatible (add columns and tables; remove them in a later release). The YAML parses and the filtered `turbo run build --filter=@matr/<app>` step was run locally; the workflow itself needs a first real run in your repository.

```
name: CI and deploy

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
```

```

cancel-in-progress: ${ github.event_name == 'pull_request' }}

env:
  TURBO_TELEMETRY_DISABLED: "1"

jobs:
  verify:
    name: Lint, typecheck, test, build
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v7
      - uses: pnpm/action-setup@v6
      - uses: actions/setup-node@v7
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm test
      - run: pnpm build

  deploy-backend:
    name: Deploy API Worker
    needs: verify
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    timeout-minutes: 10
    environment: production
    steps:
      - uses: actions/checkout@v7
      - uses: pnpm/action-setup@v6
      - uses: actions/setup-node@v7
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      # Migrations first, and written to be backward compatible, so the old
      Worker keeps
      # working until the new one is live.
      - name: Apply D1 migrations
        uses: cloudflare/wrangler-action@v4

```

```

with:
  apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
  accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
  workingDirectory: packages/backend
  command: d1 migrations apply matr-d1 --remote
- name: Deploy Worker
  uses: cloudflare/wrangler-action@v4
  with:
    apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
    accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
    workingDirectory: packages/backend
    command: deploy

```

deploy-frontends:

```

name: Deploy ${{ matrix.app }}
needs: deploy-backend
runs-on: ubuntu-latest
timeout-minutes: 10
environment: production
strategy:
  fail-fast: false
  matrix:
    app: [waitlist] # add landing, user-app and admin as they ship
steps:
  - uses: actions/checkout@v7
  - uses: pnpm/action-setup@v6
  - uses: actions/setup-node@v7
    with:
      node-version: 22
      cache: pnpm
  - run: pnpm install --frozen-lockfile
  - name: Build static export
    run: pnpm turbo run build --filter=@matr/${{ matrix.app }}
  - name: Deploy static assets to Workers
    uses: cloudflare/wrangler-action@v4
    with:
      apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
      accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
      workingDirectory: apps/${{ matrix.app }}
      command: deploy

```

Setup in GitHub: create an environment named `production` (optionally with required reviewers, which puts a manual approval in front of both deploy jobs) and add `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID` to it. Cloudflare's [GitHub Actions guide](#) recommends starting the token from the "Edit Cloudflare Workers" template and scoping it to the one account; if the migration step fails with an authorisation error, add Account, D1, Edit to the token. The action versions are the latest major tags at the time of writing.

The frontend deploy shape matches the Cloudflare documentation:

`apps/waitlist/wrangler.toml` declares an assets-only Worker, and `wrangler deploy` uploads `./out` and attaches the custom domain.

```
name = "matr-waitlist"
compatibility_date = "2026-10-01"
routes = [{ pattern = "waitlist.matr.studio", custom_domain = true }]

# Assets-only Worker: no `main`, so no Worker code ever runs for page views.
[assets]
directory = "./out"
not_found_handling = "404-page"
```

Rollback and recovery

- Worker: `pnpm --filter @matr/backend exec wrangler rollback` reverts to the previous version; pass a version ID to pick an older one.
- Database: D1 Time Travel restores to any point in the last 7 days on the free plan, with `wrangler d1 time-travel info matr-d1` to list bookmarks and `wrangler d1 time-travel restore matr-d1 --timestamp=<unix>` to restore.
- Frontends: redeploy the previous commit; static assets have no state to restore.
- Staging: the free plan allows 10 D1 databases, so a `matr-d1-staging` database and a second Worker name cost nothing extra, but staging traffic spends the same account-wide daily Worker and D1 limits as production.

7. Monitoring, rate limiting & defensive controls

The 100,000-requests/day cap is a shared pool, so a scraper or a bot signing up fake emails can exhaust it for real users; the defence is three layers, cheapest first, plus a monitor that warns at 80% because the free plan will not do that for you.

Layer	Where it runs	Free-plan limit	What it stops	Costs a Worker request?
1. WAF rate-limiting rule	Cloudflare edge, before the Worker	1 rule, per IP, 10 s window, 10 s block	Floods and scrapers	No: blocked requests never reach the Worker
2. Rate Limiting binding	Inside the Worker	5 signups/min and 60 reads/min per IP in this design	Slow, sustained abuse of one endpoint	Yes
3. Turnstile on signup	Worker calls Cloudflare siteverify	Free	Scripted signups	Yes, plus 1 subrequest

Layer 1: WAF rate-limiting rule

This is the only layer that protects the quota itself, because a request it blocks never invokes the Worker. The free plan allows [one rule](#), counted per IP over a 10-second window with a 10-second block, and it matches on a limited set of fields such as the path. In the dashboard, open Security, WAF, Rate limiting rules and create:

Setting	Value
Name	<code>api-burst-guard</code>
If incoming requests match	URI Path starts with <code>/api/</code>
Counting characteristic	IP
Requests / period	20 requests per 10 seconds
Action / duration	Block for 10 seconds

The rule applies to the zone, so it only helps on the custom domain; that is the second reason `workers_dev` is disabled in production (section 6). A 20-per-10-seconds ceiling is far above what a person filling a form can produce, but shared office or campus IPs can hit

it, so raise it if support reports false blocks. If the free-plan editor offers only a limited set of operators, use "URI Path contains /api/"; the intent is simply to cover the API paths and nothing else.

Layer 2: Rate Limiting binding

The two `[[ratelimits]]` blocks in `wrangler.toml` and the `rateLimit()` middleware from section 4 already implement this layer. Period must be 10 or 60 seconds, and the key is path plus client IP, so a burst on one endpoint does not lock a visitor out of the others. Cloudflare's [binding docs](#) do not state plan restrictions; the binding works in `wrangler dev` and returned 429 after five signups in a minute during testing, and it should be confirmed once on the deployed Worker. The docs describe the binding as permissive and eventually consistent, with a separate counter per key in each Cloudflare location, and say it is not meant as an accurate accounting system, so treat the limits as guard rails rather than exact quotas. The docs also advise against IP addresses as keys, because many people share one IP on mobile networks and behind proxies. Anonymous signup has no better identifier, so this design keys on IP and keeps the signup limit (5 per minute) generous enough for a shared address; once ``user-app`` has sessions, key the authenticated routes on the user ID instead.

This design ships no in-memory limiter, on purpose. A `Map` in module scope lives only as long as one isolate: Cloudflare runs many isolates per location, recycles them freely, and shares no memory between them, so a per-isolate counter resets unpredictably and undercounts under exactly the load you care about. If a Map-based pre-filter is ever added, cap its size (for example 5,000 keys, cleared when full) to stay well inside the 128 MB isolate limit, and keep layers 1 and 2 as the real controls.

Layer 3: Turnstile and request hygiene

The signup route already verifies a [Turnstile](#) token whenever the `TURNSTILE_SECRET` secret exists. To switch it on, create a Turnstile widget for `waitlist.matr.studio`, set the secret with `wrangler secret put TURNSTILE_SECRET`, and render the widget in `waitlist-form.tsx`, sending its token as `turnstileToken` in the `join()` call. The form in section 5 does not render the widget yet; until it does, leave the secret unset or signups will be rejected.

The remaining controls are already in the code from section 4: a 4 KB body cap, strict zod validation, an identical 202 response whether or not an email already exists, an exact-match CORS allow-list (a browser-side control only; it does not stop scripts), and

`workers_dev` disabled. Add `Retry-After` handling to the form if rate-limit rejections become common.

Quota alerts on the free plan

Cloudflare's usage-threshold notification is documented as [available to pay-as-you-go accounts only](#), so a free account has no built-in "80% of quota" email. The substitute is a cron-triggered check inside the API Worker: it asks Cloudflare's GraphQL Analytics API for today's usage every 30 minutes and posts to a Slack or Discord webhook the first time a metric crosses 80% and again at 95%. De-duplication uses one row per day, metric and threshold in `ops_alerts` (migration 0002), so the worst case is 6 D1 writes and 48 cron runs a day.

Setup:

1. Create an API token with a single permission, Account, Account Analytics, Read, and store it as `CF_API_TOKEN` (and your account ID as `CF_ACCOUNT_ID`).
2. Create an incoming webhook in Slack or Discord and store it as `ALERT_WEBHOOK_URL`.
3. Deploy. The `[triggers]` block in `wrangler.toml` registers the `*/30 * * * *` schedule.

`packages/backend/src/cron/usage-monitor.ts`:

```
import type { Bindings } from "../env";

// Workers Free limits (daily, reset 00:00 UTC). Keep in sync with the quota
// table in the docs.
const LIMITS = { requests: 100_000, rowsRead: 5_000_000, rowsWritten: 100_000
} as const;
const LEVELS = [80, 95] as const;

const QUERY = /* GraphQL */ `
  query Usage($accountTag: string!, $start: string!, $end: string!, $day:
string!) {
    viewer {
      accounts(filter: { accountTag: $accountTag }) {
        workersInvocationsAdaptive(limit: 1000, filter: { datetime_geq:
$start, datetime_leq: $end }) {
          sum { requests }
        }
        d1AnalyticsAdaptiveGroups(limit: 1000, filter: { date_geq: $day,
date_leq: $day }) {
```

```

        sum { rowsRead rowsWritten }
      }
    }
  }
};

type UsageResponse = {
  data?: {
    viewer: {
      accounts: Array<{
        workersInvocationsAdaptive: Array<{ sum: { requests: number } }>;
        d1AnalyticsAdaptiveGroups: Array<{ sum: { rowsRead: number;
rowsWritten: number } }>;
      }>;
    };
  };
  errors?: unknown[];
};

async function fetchUsage(env: Bindings, now: Date) {
  const day = now.toISOString().slice(0, 10);
  const res = await fetch("https://api.cloudflare.com/client/v4/graphql", {
    method: "POST",
    headers: { Authorization: `Bearer ${env.CF_API_TOKEN}`, "Content-Type":
"application/json" },
    body: JSON.stringify({
      query: QUERY,
      variables: {
        accountTag: env.CF_ACCOUNT_ID,
        start: `${day}T00:00:00Z`,
        end: now.toISOString(),
        day,
      },
    }),
  });
  if (!res.ok) throw new Error(`GraphQL usage query failed with HTTP
${res.status}`);
  const json = (await res.json()) as UsageResponse;
  const account = json.data?.viewer.accounts[0];
  if (!account) throw new Error(`GraphQL usage query failed:
${JSON.stringify(json.errors)}`);

```

```

    const sum = <T>(rows: T[], pick: (r: T) => number) => rows.reduce((n, r) =>
n + pick(r), 0);
    return {
      day,
      usage: {
        requests: sum(account.workersInvocationsAdaptive, (r) =>
r.sum.requests),
        rowsRead: sum(account.d1AnalyticsAdaptiveGroups, (r) =>
r.sum.rowsRead),
        rowsWritten: sum(account.d1AnalyticsAdaptiveGroups, (r) =>
r.sum.rowsWritten),
      },
    };
  }
}

```

/** Runs from the cron trigger. Alerts once per (day, metric, threshold) via a Slack/Discord-style webhook. */

```

export async function checkUsage(env: Bindings): Promise<void> {
  const { day, usage } = await fetchUsage(env, new Date());

  for (const metric of Object.keys(LIMITS) as Array<keyof typeof LIMITS>) {
    const pct = (usage[metric] / LIMITS[metric]) * 100;
    for (const level of LEVELS) {
      if (pct < level) continue;

      // INSERT OR IGNORE: changes === 1 only the first time this threshold
is crossed today.
      const res = await env.DB.prepare(
        "INSERT OR IGNORE INTO ops_alerts (day, metric, level) VALUES (?1, ?
2, ?3)",
      )
        .bind(day, metric, level)
        .run();
      if (res.meta.changes !== 1) continue;

      const text = `Matr Studio: ${metric} at ${pct.toFixed(0)}% of the free
daily limit (${usage[metric].toLocaleString("en-US")} /
${LIMITS[metric].toLocaleString("en-US")}). Resets 00:00 UTC.`;
      await fetch(env.ALERT_WEBHOOK_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ content: text, text }), // Discord reads
"content", Slack reads "text"

```

```
    });  
  }  
}  
}
```

To test the handler locally, run `pnpm dev` (the dev script passes `--test-scheduled`) and call `curl http://localhost:8787/cdn-cgi/handler/scheduled`. With the placeholder credentials from `.dev.vars` it fails at the GraphQL call and logs the error, which confirms the trigger fires; with a real analytics token and a webhook URL it exercises the whole path.

What was and was not verified: the handler compiles and the trigger runs locally. The GraphQL query was not run against Cloudflare’s live API from the environment this document was prepared in. The dataset and field names (`workersInvocationsAdaptive`, `d1AnalyticsAdaptiveGroups`, `rowsRead`, `rowsWritten`) come from Cloudflare’s [Workers metrics](#) and [D1 metrics](#) docs, and the variable declarations follow the lower-case style those docs use. Paste the query into the GraphQL explorer once with your token before relying on it. Analytics data lags real usage by a few minutes, which is why the thresholds are 80% and 95% rather than 99%.

Two things stay manual. R2 usage is not in the automated check (its dataset was not verified), and R2 has no hard cap: review R2, Metrics in the dashboard weekly while uploads are live. And the dashboard graphs, Workers and Pages, `matr-api`, Metrics (requests, errors, CPU-time quantiles) and D1, `matr-d1`, Metrics (rows read and written), are the place to look when an alert fires.

When an alert fires

Alert	First checks	Levers
Requests at 80%	Is traffic real? Look at the request graph and the top paths	Tighten the WAF rule, enable Turnstile, add a longer browser <code>max-age</code> on cacheable GETs, consider a same-origin route to drop preflights
Rows written at 80%	Fake signups? Compare <code>created_at</code> clustering in <code>waitlist_subscribers</code>	Turnstile, a lower signup limit, drop the stats trigger to save 1 write per signup

Alert	First checks	Levers
Rows read at 80%	Uncached full-table queries or a hot admin screen	Longer cache TTL, a counter table instead of <code>COUNT(*)</code> , an index on the filtered column
Any metric at 95%	Decide whether to protect the rest of the day	Add a temporary WAF custom rule that blocks <code>/api/waitlist</code> , then remove it after the 00:00 UTC reset